Optimization for Deep Learning

SGD, Momentum, Adam, and Learning-Rate Schedules

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

From backprop to optimization

Where we are

Backprop gave us one thing: the **gradient** of the loss.

$$g_t = \nabla_\theta \mathcal{L}(\theta_t)$$

But a gradient is only a **direction**. Training still has to decide:

How do we turn a stream of gradients into good parameters? That is the job of the **optimizer**.

The spine of this lecture

Backprop gives gradients;

optimization decides how to use them.

each step: current params → gradient → update rule → new params

The optimizer is a small function

Every method in this lecture has the same skeleton:

$$\theta_{t+1} = \mathrm{update}(\theta_t, g_t)$$

They differ only in **what state they keep** and **how they scale the step**:

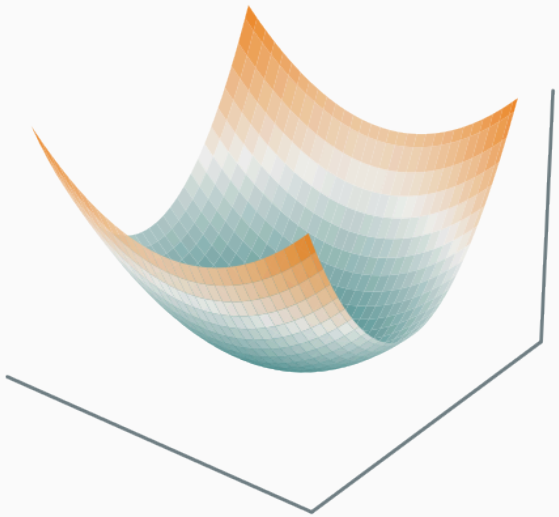
- plain GD keeps **nothing** — just $-\eta g_t$;
- momentum keeps a **running direction**;
- Adam keeps a running **direction** and a running **magnitude**.

The loss landscape

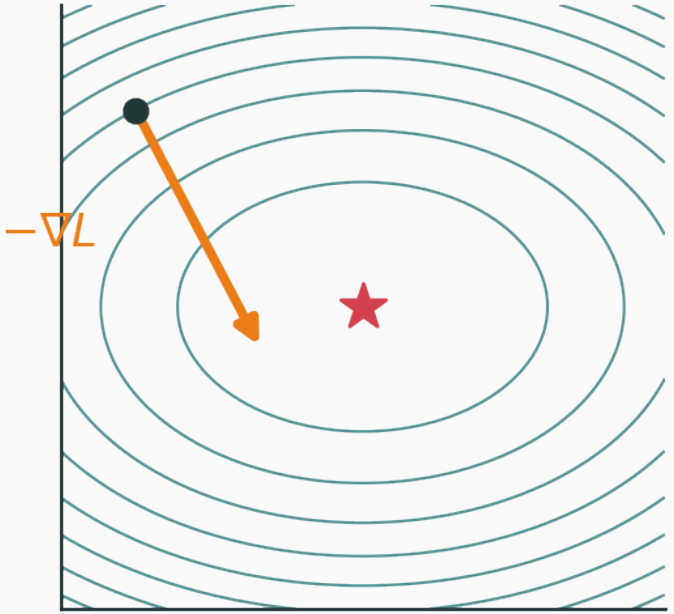
V VISUAL

Training = descending a surface $\mathcal{L}(\theta)$ over millions of parameters.

loss surface $\mathcal{L}(\theta_1, \theta_2)$



contours + gradient step



the gradient points **uphill**; we step along $-\nabla\mathcal{L}$, perpendicular to contours

Gradient descent from a Taylor series

D DERIVATION

Expand the loss around the current point θ for a small step Δ:

$$\mathcal{L}(\theta + \Delta) \approx \mathcal{L}(\theta) + \nabla \mathcal{L}(\theta)^\top \Delta + \frac{1}{2} \Delta^\top H \Delta$$

Keep only the **first-order** term (small steps $\Longrightarrow$ curvature negligible):

$$\mathcal{L}(\theta + \Delta) \approx \mathcal{L}(\theta) + \nabla \mathcal{L}(\theta)^\top \Delta$$

Steepest descent falls out

D DERIVATION

Minimize the linear model over a **trust region** $\|\Delta\| \le r$:

$$\min_{\|\Delta\| \le r} \nabla \mathcal{L}(\theta)^\top \Delta$$

By Cauchy–Schwarz this is most negative when Δ is **anti-parallel** to the gradient:

$$\Delta^\star = -r \frac{\nabla \mathcal{L}(\theta)}{\|\nabla \mathcal{L}(\theta)\|} \quad \propto \quad -\nabla \mathcal{L}(\theta)$$

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t)$$

What is the learning rate, really?

D DERIVATION

Folding the trust-region radius into a single step size gives η. Compare with the **second-order** (Newton) step that also uses curvature:

$$\Delta_{\text{Newton}} = -H^{-1}\nabla\mathcal{L}$$

η plays the role of an **inverse-curvature** / trust-region size. A scalar η is a crude stand-in for H^{-1} — too big overshoots sharp directions, too small crawls on flat ones.

Basic gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t)$$

- η — the **learning rate** (step size);
- $\nabla \mathcal{L}$ — the direction of steepest ascent, so $-\nabla \mathcal{L}$ descends;
- one hyperparameter, no memory — the simplest optimizer there is.

Worked example: 1-D quadratic

D DERIVATION

Minimum is at $\theta = 3$. Set up the loss, gradient, and starting point:

$$\mathcal{L}(\theta) = (\theta - 3)^2, \quad \nabla\mathcal{L} = 2(\theta - 3), \quad \theta_0 = 0, \quad \eta = 0.1$$

Step 1 — plug $\theta_0 = 0$ into the update rule:

$$\theta_1 = 0 - 0.1 \cdot 2(0 - 3)$$

$$\theta_1 = 0 - 0.1 \cdot (-6) = 0.6$$

the gradient -6 points left of the min, so the step moves us **right**, toward 3.

Worked example: 1-D quadratic (cont.)

D DERIVATION

Recall $\theta_1 = 0.6$, still with $\eta = 0.1$ on $\mathcal{L} = (\theta - 3)^2$. **Step 2** — gradient at 0.6 is $2(0.6 - 3) = -4.8$:

$$\theta_2 = 0.6 - 0.1 \cdot (-4.8) = 1.08$$

Step 3 — gradient at 1.08 is $2(1.08 - 3) = -3.84$:

$$\theta_3 = 1.08 - 0.1 \cdot (-3.84) = 1.464$$

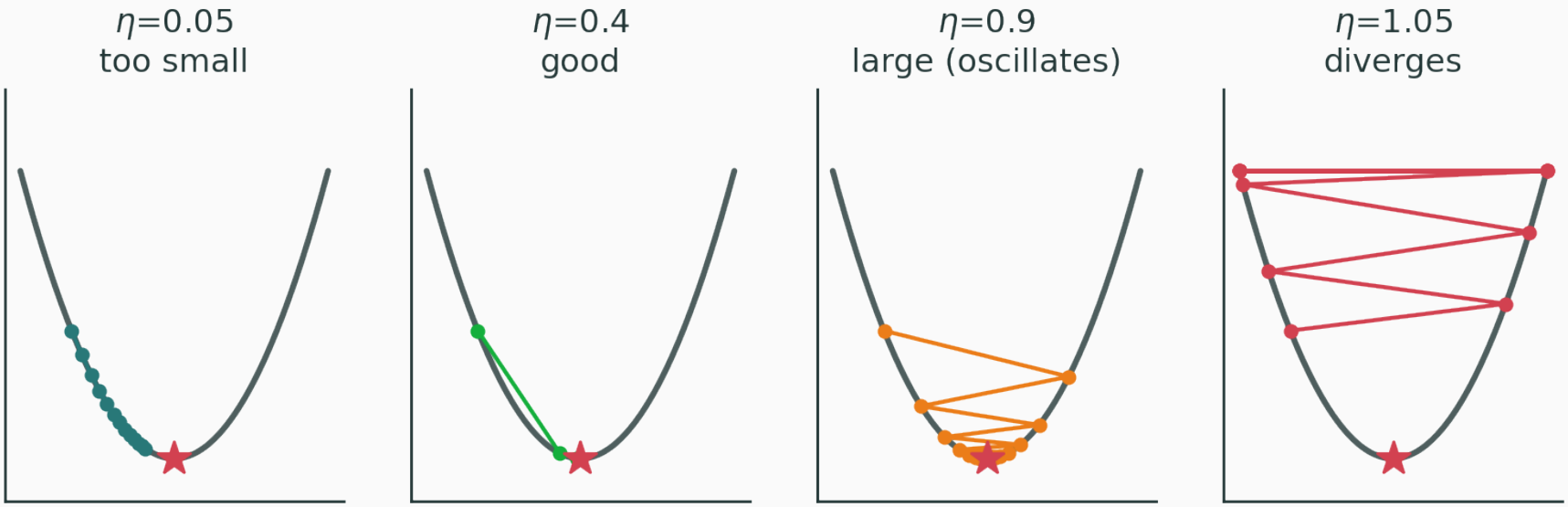
$$0 \to 0.6 \to 1.08 \to 1.464 \to \ldots \to 3$$ — each step covers 80% of the remaining gap.

The distance to the minimum obeys $(\theta_t - 3) \to (1 - 2\eta)(\theta_t - 3)$; here $1 - 2\eta = 0.8$, a clean geometric decay.

The learning rate matters

V VISUAL

Same curve $\mathcal{L} = (\theta - 3)^2$, four choices of η:



too small crawls · good converges fast · large oscillates · $\eta > 1$ diverges

Interactive: optimizer race

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/optimizer-race

Drop a start point on a loss surface and watch gradient descent trace its path as you sweep the learning rate — see crawl, converge, oscillate, and diverge live.

Try values around the stability boundary and watch the error multiplier $1 - 2\eta$.

Checkpoint: learning-rate stability

Q QUESTION

For $\mathcal{L} = (\theta - 3)^2$, gradient descent diverges for which learning rates?

A. $\eta < 0$ only

B. $0 < \eta < 0.5$

C. $0.5 < \eta < 1$

D. $\eta > 1$

Answer: learning-rate stability

A ANSWER

Correct: D — $\eta > 1$

Why: The error is multiplied by $1 - 2\eta$ each step. Convergence requires its magnitude to be below one; beyond $\eta = 1$ it grows.

Full-batch vs stochastic gradient descent

The loss is an average

Training loss is an **empirical risk** — a mean over n examples:

$$\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(\theta), \qquad \nabla\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \nabla\ell_i(\theta)$$

The true gradient needs a pass over the **entire dataset**. For $n = 10^6$ that is a million backprops per step.

Full-batch gradient descent

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\theta_t)$$

Pro — the exact gradient; smooth, deterministic descent.

Con — one step costs a **full epoch**; far too slow for large n.

Stochastic gradient descent (SGD)

Sample one example $I \sim \mathrm{Uniform}(1, \ldots, n)$ and step on **its** gradient:

$$g_t = \nabla \ell_I(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta\, g_t$$

one example per step — up to $n \times$ more updates per epoch

SGD is an unbiased estimator

D DERIVATION

Take the expectation over the random index I:

$$\mathbb{E}_I[g_t] = \sum_{i=1}^{n} \Pr(I = i)\, \nabla \ell_i(\theta_t) = \sum_{i=1}^{n} \frac{1}{n} \nabla \ell_i(\theta_t)$$

$$\mathbb{E}_I[g_t] = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\theta_t) = \nabla \mathcal{L}(\theta_t)$$

on average, SGD steps in the true gradient direction

Unbiased, but noisy

Write the stochastic gradient as the truth plus zero-mean noise:

$$g_t = \nabla \mathcal{L}(\theta_t) + \varepsilon_t, \quad \mathbb{E}[\varepsilon_t] = 0, \quad \mathrm{Var}(\varepsilon_t) > 0$$

Each step is right **on average** but wrong **individually**. The path jitters — that noise both hurts (never fully settles) and helps (escapes bad regions).

Minibatch SGD — the practical middle

Average the gradient over a **batch** B_t of B examples:

$$g_t = \frac{1}{B} \sum_{i \in B_t} \nabla \ell_i(\theta_t)$$

Still unbiased, $\mathbb{E}[g_t] = \nabla \mathcal{L}$, but the noise shrinks:

$$\mathrm{Var}(g_t) \approx \frac{1}{B} \, \mathrm{Var}(\nabla \ell_i)$$

larger batch $\Rightarrow$ $\sqrt{B}$ times less gradient noise

The batch-size tradeoff

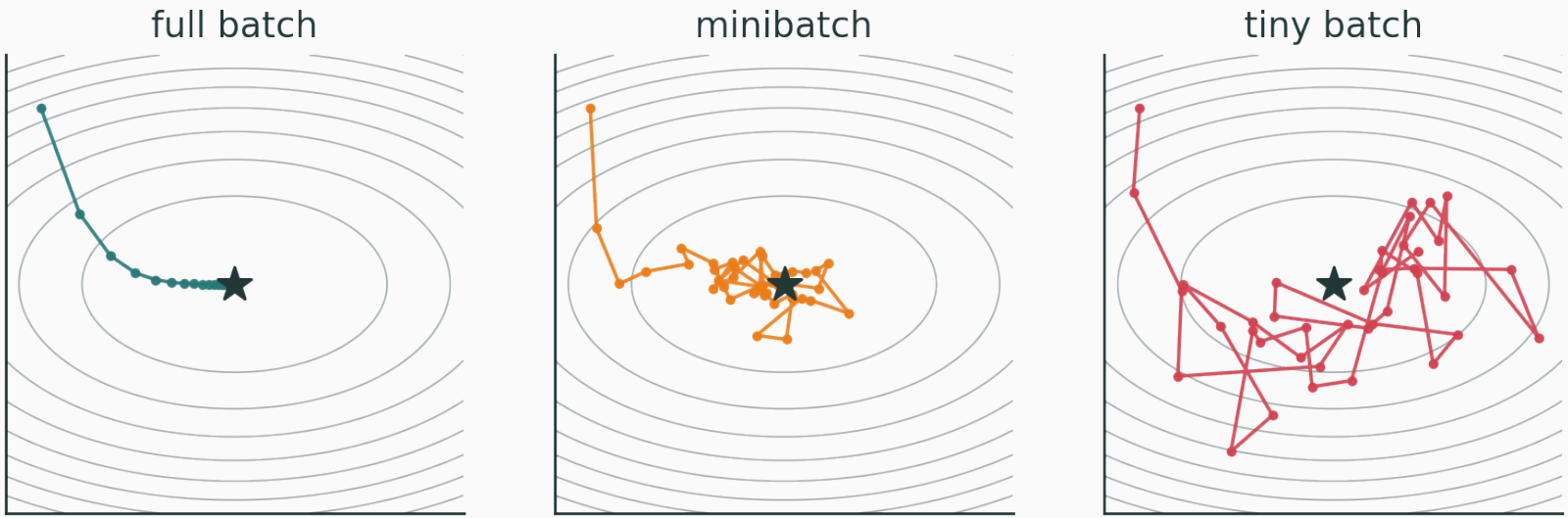
Batch size	Gradient	Behaviour
small (1–32)	very noisy	cheap steps, explores, can escape saddles
medium (64–512)	moderate	the usual sweet spot
large (4k+)	near-exact	stable but costly; needs LR tuning

Batch size is set as much by **GPU memory and throughput** as by optimization.

Batch size shapes the noise

V VISUAL

Same bowl, three noise levels (full / mini / tiny batch):



full batch glides · minibatch jitters toward the minimum · tiny batch bounces

Interactive: SGD noise

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/optimizer-race

Switch between full-batch and stochastic updates and watch the trajectory go from a smooth glide to a noisy walk as the batch size shrinks.

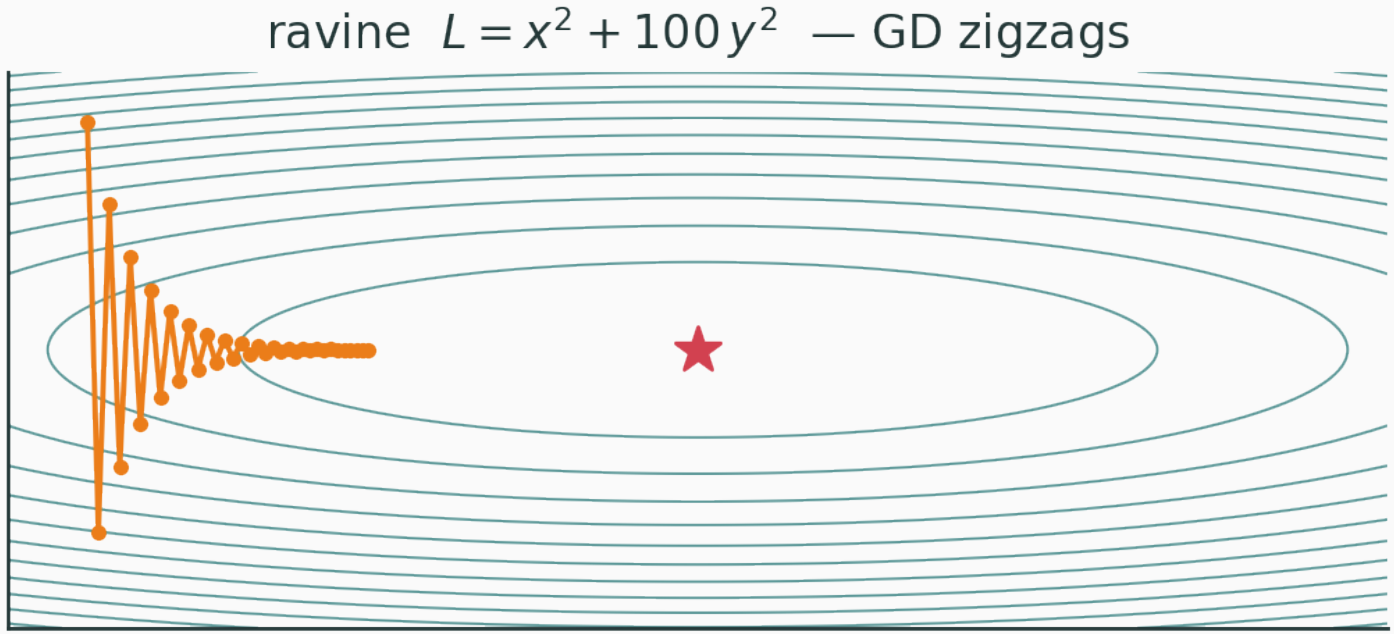
Compare a smooth full-batch path with the small random deviations of minibatches.

Momentum

The problem: ravines

V VISUAL

Ill-conditioned loss $\mathcal{L} = x^2 + 100y^2$: steep across, shallow along.



GD must use a **tiny** η to stay stable in y — so it **zigzags** and crawls in x

The idea: keep a velocity

Accumulate past gradients into a **velocity**, then step along it:

$$v_t = \beta\, v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta\, v_t$$

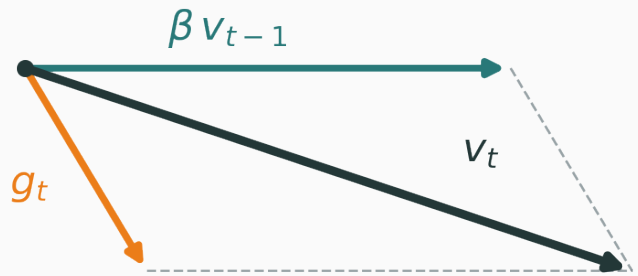
- $\beta \in [0, 1)$ — the **momentum** coefficient (typically 0.9);
- v_t is a running memory of **where we have been going**.

Momentum as a vector sum

V VISUAL

The update is a **vector addition**: the decayed velocity plus this step's gradient.

$v_t = \beta\, v_{t-1} + g_t$ — momentum adds this step to the running velocity



the long βv_{t-1} dominates; the cross-valley g_t is largely cancelled step to step

Momentum is an exponential moving average

D DERIVATION

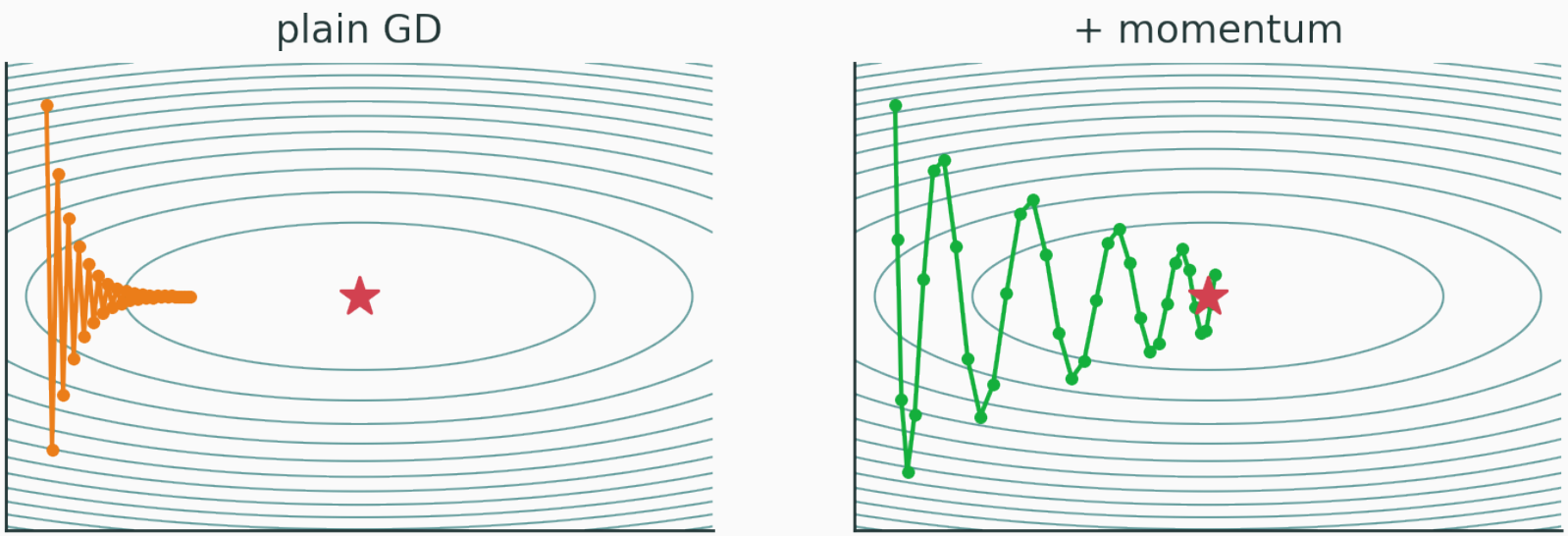
Unroll the recursion $v_t = \beta v_{t-1} + g_t$:

$$v_t = g_t + \beta g_{t-1} + \beta^2 g_{t-2} + \beta^3 g_{t-3} + \dots$$

A **geometrically weighted sum** of all past gradients. Recent gradients dominate; old ones fade by β each step. It **remembers the direction**.

Why momentum helps

V VISUAL



- **oscillating** directions (y) → gradients flip sign → they **cancel** in v_t;
- **consistent** directions (x) → gradients agree → they **accumulate** and accelerate;
- averaging also **smooths** stochastic noise.

Worked example: the EMA

D DERIVATION

Use the **normalized** form $m_t = \beta m_{t-1} + (1-\beta) g_t$ with $\beta = 0.9$, start $m_0 = 0$, gradients $g = (10, 8, 11)$: Step 1 — take 10% of the new gradient, keep 90% of the old average:

$$m_1 = 0.9 \cdot 0 + 0.1 \cdot 10 = 1.0$$

one gradient of 10 only nudges the average to 1.0 — the memory of $m_0 = 0$ still dominates.

Worked example: the EMA (cont.)

D DERIVATION

Carry $m_1 = 1.0$ forward; $\beta = 0.9$, next gradients 8 then 11. **Step 2** — carry $0.9 \cdot 1.0$ forward, add $0.1 \cdot 8$:

$$m_2 = 0.9 \cdot 1.0 + 0.1 \cdot 8 = 0.9 + 0.8 = 1.7$$

Step 3 — carry $0.9 \cdot 1.7$, add $0.1 \cdot 11$:

$$m_3 = 0.9 \cdot 1.7 + 0.1 \cdot 11 = 1.53 + 1.1 = 2.63$$

raw gradients average ~ 9.7, but starting from 0 the EMA is still climbing at $m_3 = 2.63$ — early averages are biased **low** (Adam will correct this).

How much does momentum remember?

The weights β^k decay geometrically; the **effective memory** is

$$\frac{1}{1-\beta} \quad \text{steps.}$$

β	effective window
0.9	~ 10 steps
0.99	~ 100 steps
0.5	~ 2 steps

Higher β = longer memory = more smoothing, but slower to react to a real change of direction.

Interactive: momentum

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/optimizer-race

Turn momentum on and off in the ravine and sweep β — watch zigzags collapse into a smooth, accelerating glide down the valley.

Adaptive learning rates

One global η is too blunt

Different parameters live on different scales and see different gradient sizes.

- a rarely-active feature gets **tiny, infrequent** gradients — wants a **big** step;
- a hot feature gets **large, frequent** gradients — wants a **small** step.

give every coordinate its own effective learning rate

The adaptive form

Scale each coordinate j by the size of **its own** recent gradients:

$$\theta_{t+1,j} = \theta_{t,j} - \frac{\eta}{\sqrt{s_{t,j}} + \varepsilon} g_{t,j}$$

- $s_{t,j}$ — a running estimate of the **squared** gradient in coordinate j;
- ε — a small constant ($\sim 10^{-8}$) to avoid dividing by zero.

RMSProp: an EMA of squared gradients

Keep an exponential average of g^2, **elementwise**:

$$s_t = \rho\, s_{t-1} + (1-\rho)\, g_t^2$$

$\sqrt{s_t}$ is the **root-mean-square** of recent gradients — hence the name **RMSProp**.

Big recent gradients $\Rightarrow$ large $\sqrt{s_t} \Rightarrow$ **smaller** step. Persistently small gradients $\Rightarrow$ **larger** step. The step self-normalizes.

Why divide by the RMS?

Dividing by $\sqrt{s_t}$ makes the update **scale-invariant** per coordinate:

$$\frac{g_{t,j}}{\sqrt{s_{t,j}}} \approx \pm 1$$

Steep and shallow directions get **comparable** step sizes — exactly what the ravine needed, without hand-tuning per axis.

Worked example: RMSProp on two axes

D DERIVATION

Steep axis sees $g = 10$, shallow axis sees $g = 0.1$; take $\rho = 0.9$, $s_0 = 0$:

$$s_{\text{steep}} = 0.9 \cdot 0 + 0.1 \cdot 10^2 = 10, \qquad s_{\text{shallow}} = 0.9 \cdot 0 + 0.1 \cdot 0.1^2 = 0.001$$

Divide each gradient by its own $\sqrt{s}$ (ignore ε):

$$\frac{g_{\text{steep}}}{\sqrt{s_{\text{steep}}}} = \frac{10}{\sqrt{10}} \approx 3.16, \qquad \frac{g_{\text{shallow}}}{\sqrt{s_{\text{shallow}}}} = \frac{0.1}{\sqrt{0.001}} \approx 3.16$$

a $100\times$ gap in raw gradient becomes the same effective step

self-normalization: both axes move at rate $\sim \eta$, so the ravine no longer zigzags.

Adam

Adam = momentum + adaptivity

Keep **two** moving averages of the gradient:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \qquad (\text{1st moment — direction})$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \qquad (\text{2nd moment — magnitude})$$

m_t **= smoothed direction** · v_t **= smoothed scale**

A subtle bug: initialization bias

D DERIVATION

We start $m_0 = 0$ and $v_0 = 0$. After one step with constant g:

$$m_1 = (1 - \beta_1)g = 0.1g \qquad (\text{way below} \quad g)$$

Early moving averages are biased **toward zero** — worst at $t = 1$, since there is nothing yet to average.

Bias correction

Divide out the shrinkage factor $1 - \beta^t$:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Then take the RMSProp-style step with the **corrected** moments:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

as $t \to \infty$, $\beta^t \to 0$ and the correction fades away.

Worked example: Adam step 1

D DERIVATION

$g_1 = 4, \quad \beta_1 = 0.9, \beta_2 = 0.999, \eta = 0.001$, start $m_0 = v_0 = 0$: **1st moment** — EMA of the gradient (direction):

$$m_1 = 0.9 \cdot 0 + 0.1 \cdot 4 = 0.4$$

2nd moment — EMA of the squared gradient (magnitude):

$$v_1 = 0.999 \cdot 0 + 0.001 \cdot 16 = 0.016$$

both are far **below** the true scale ($g = 4$, $g^2 = 16$) — biased toward 0 by the cold start.

Worked example: Adam step 1 (cont.)

D DERIVATION

Undo the bias by dividing out $1 - \beta^t$ (here $t = 1$):

$$\hat{m}_1 = \frac{0.4}{1 - 0.9} = 4$$

$$\hat{v}_1 = \frac{0.016}{1 - 0.999} = 16$$

Now take the RMSProp-style step with the corrected moments:

$$\Delta\theta_1 = -\eta \frac{\hat{m}_1}{\sqrt{\hat{v}_1}} = -0.001 \cdot \frac{4}{\sqrt{16}} = -0.001$$

bias correction rescues m_1, v_1 back to the true gradient scale — the step is exactly $-\eta$.

Worked example: Adam step 2

D DERIVATION

$g_2 = 6$ (now $t = 2$), carrying $m_1 = 0.4$, $v_1 = 0.016$ from before: **1st moment** — decay the old average, add 10% of the new gradient:

$$m_2 = 0.9 \cdot 0.4 + 0.1 \cdot 6 = 0.96$$

2nd moment — same, with the squared gradient 36:

$$v_2 = 0.999 \cdot 0.016 + 0.001 \cdot 36 = 0.051984$$

the bias factor is now $1 - \beta^2$, weaker than at $t = 1$ — the correction is already fading.

Worked example: Adam step 2 (cont.)

D DERIVATION

Bias-correct with $t = 2$, so divide by $1 - \beta^2$:

$$\hat{m}_2 = \frac{0.96}{1 - 0.9^2} = \frac{0.96}{0.19} \approx 5.05$$

$$\hat{v}_2 = \frac{0.051984}{1 - 0.999^2} \approx 26.0$$

$$\Delta\theta_2 = -0.001 \cdot \frac{5.05}{\sqrt{26.0}} \approx -0.000991$$

each coordinate moves by roughly $\pm\eta$, regardless of the raw gradient size.

Adam intuition

$$\theta_{t+1} = \theta_t - \eta \frac{\overbrace{\hat{m}_t}^{\text{which way}}}{\underbrace{\left(\sqrt{\hat{v}_t} + \varepsilon\right)}_{\text{how big}}}$$

- numerator $\hat{m}_t$ — a **momentum-smoothed direction**;
- denominator $\sqrt{\hat{v}_t}$ — a **per-coordinate magnitude** that self-normalizes;
- together: a smoothed, scale-free step in every coordinate.

Adam defaults

Hyperparameter	Default
β_1 (momentum)	0.9
β_2 (RMS)	0.999
ε	10^{-8}
η (learning rate)	10^{-3} (still tune this)

These defaults "just work" on a huge range of problems — a big reason Adam is the default optimizer.

Checkpoint: Adam’s two memories

Q QUESTION

What extra information does Adam keep beyond plain gradient descent?

A. Only the current loss value

B. A first-moment direction and a second-moment magnitude estimate

C. A full Hessian matrix

D. The complete training dataset

Answer: Adam’s two memories

A ANSWER

Correct: B — A first and a second moment

Why: The first moment smooths direction like momentum; the second tracks per-coordinate gradient scale for adaptive steps.

Interactive: Adam vs the rest

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/optimizer-race

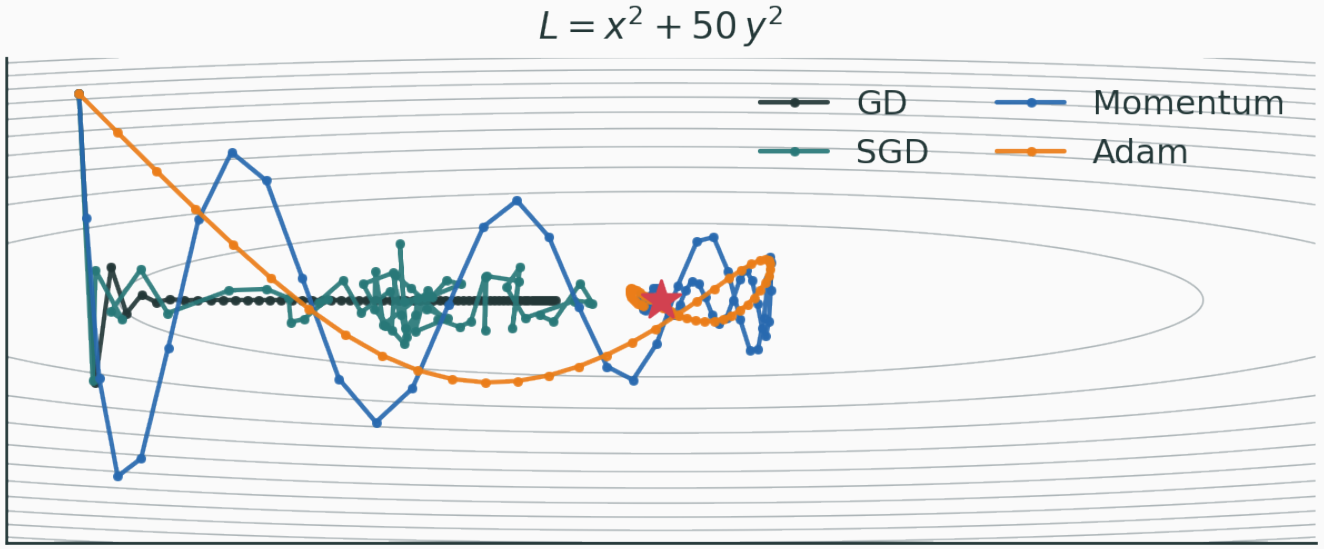
Race GD, SGD, momentum and Adam on the same surface. Watch Adam’s per-coordinate scaling walk straight down a ravine that makes plain GD zigzag.

Putting the optimizers side by side

Same loss, four optimizers

V VISUAL

$\mathcal{L} = x^2 + 50y^2$, identical start point:



GD crawls · SGD jitters · momentum overshoots then settles · Adam cuts across

Summary table

Optimizer	Memory	Adaptive?	Strength
GD	none	no	exact gradient, stable
SGD	none	no	cheap, noisy, explores
Momentum	1st moment	no	smooths valleys, accelerates
Adam	1st + 2nd	yes	robust default, per-coord scaling

When to use what

- **Adam / AdamW** — the default. Fast, robust, forgiving of LR. Transformers, most research.
- **SGD + momentum** — often the **best final generalization**, especially in vision (ResNets, ConvNets) — but needs more LR tuning.
- **Either way, the LR schedule matters** as much as the optimizer choice.

Pragmatic rule: start with AdamW + a cosine schedule; reach for SGD+momentum when you are chasing the last bit of test accuracy.

AdamW: decoupled weight decay

★ OPTIONAL

Classic Adam folds ℓ_2 regularization into the gradient — which then gets **rescaled** by $\sqrt{\hat{v}}$, distorting it. AdamW instead applies weight decay **directly** to the parameters:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} - \eta\lambda\,\theta_t$$

decoupled decay ⟹ the modern default for training large models

Learning-rate schedules

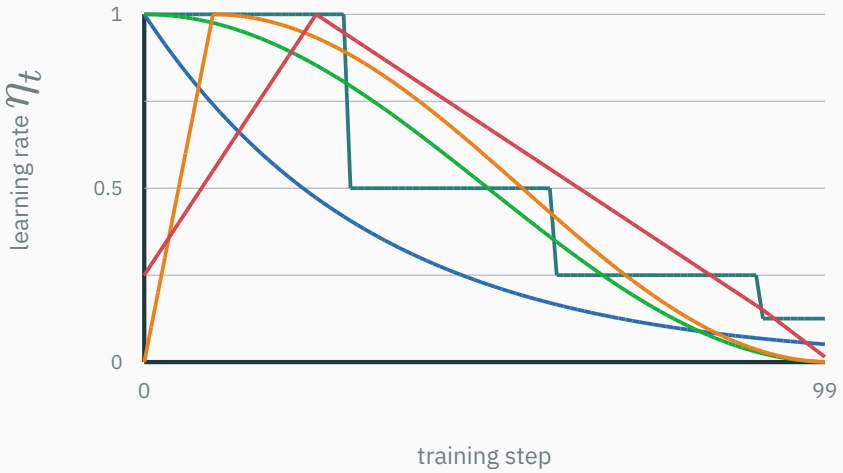
A fixed learning rate is rarely ideal

- **early** training: far from any minimum → want **large** steps to make progress;
- **late** training: near a minimum → want **small** steps to settle, not bounce.

so decay the learning rate over training

Common schedules

V VISUAL



step · exponential · cosine · warmup+cosine · one-cycle

Warmup

Start from **near zero** and ramp up over the first T_{warmup} steps, then decay:

$$\eta_t = \eta_{\max} \cdot \frac{t}{T_{\text{warmup}}} \quad \text{for } t \leq T_{\text{warmup}}$$

Early gradients are large and unreliable (moments not yet warmed up); a big LR then can **blow up** training. Warmup is near-essential for Transformers.

Cosine decay + warmup — the modern default

$$\eta_t = \frac{1}{2}\eta_{\max}\left(1 + \cos\left(\pi \frac{t - T_{\text{warmup}}}{T - T_{\text{warmup}}}\right)\right)$$

- smooth decay from $\eta_{\max}$ down to ~ 0;
- no sharp drops (unlike step decay);
- pairs naturally with a short linear warmup.

Interactive: LR schedules

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/lr-schedule-visualizer

Compose warmup length, peak LR, and decay shape (step / exponential / cosine / one-cycle) and watch the resulting η_t curve — and how it changes the loss trajectory.

Optimization pathologies

Too much gradient noise

SGD noise helps **explore** — but past a point it **prevents convergence**.

$$\theta_{t+1} = \theta_t - \eta(\nabla \mathcal{L} + \varepsilon_t)$$

If $\eta \operatorname{Var}(\varepsilon_t)$ stays large, the iterate **bounces around** the minimum forever.
Remedies: decay η, grow the batch, or use momentum to average the noise out.

Vanishing and exploding gradients

Backprop multiplies Jacobians across layers:

$$\nabla_{\theta_1} \mathcal{L} = \prod_{\ell} J_\ell \cdot (\ldots)$$

Vanishing — factors $< 1 \Rightarrow$ product $\to 0 \Rightarrow$ early layers stop learning.

Exploding — factors $> 1 \Rightarrow$ product blows up $\Rightarrow$ NaNs / divergence.

fixes live in **architecture**: activation choice, init, normalization, residual connections, gradient clipping.

Saddle points

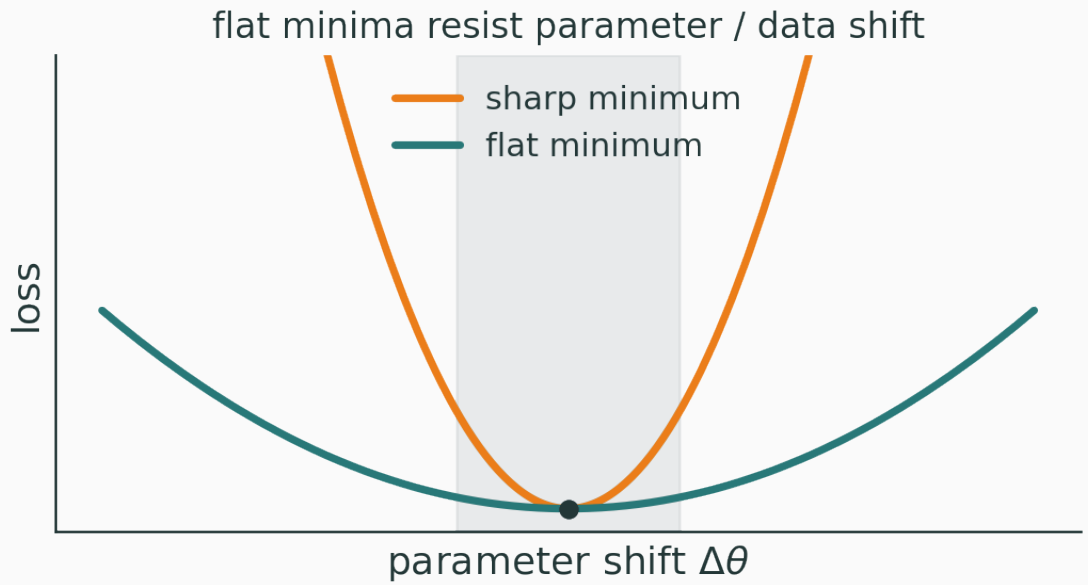
In high dimensions, most points with $\nabla \mathcal{L} = 0$ are **saddles**, not minima — the Hessian has **mixed-sign** eigenvalues.

Plain gradient descent can **stall** on the flat directions of a saddle. SGD's noise is a feature here: it **kicks the iterate off** the saddle and lets training continue.

Sharp vs flat minima

V VISUAL

★ OPTIONAL



Flat minima are widely associated with **better generalization**: a small shift in parameters or data barely changes the loss. Small-batch SGD tends to prefer flatter minima. (an active, subtle research area)

The training loop, end to end

One optimization step in PyTorch

I INTERACTIVE

```
for x, y in loader:                  # minibatch
    optimizer.zero_grad()            # clear old gradients
    logits = model(x)                # forward pass
    loss   = loss_fn(logits, y)      # scalar loss
    loss.backward()                  # backprop -> param.grad
    optimizer.step()                 # update rule uses grads
```

forward → loss → **backprop (gradients)** → **optimizer (update)** — every lecture so far, in six lines.

The step, three ways

V VISUAL

```
# SGD
p -= lr * p.grad

# SGD + momentum
v = beta * v + p.grad
p -= lr * v

# Adam (per parameter)
m = b1*m + (1-b1)*p.grad
v = b2*v + (1-b2)*p.grad**2
mhat = m/(1-b1**t); vhat = v/(1-b2**t)
p -= lr * mhat / (vhat.sqrt() + eps)
```

same loop — swap only what `optimizer.step()` does.

Choosing an optimizer in one line

```
opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)
opt = torch.optim.AdamW(model.parameters(), lr=3e-4)

sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
```

The optimizer and the schedule are **two knobs**; both matter. Defaults above are sane starting points.

One-slide summary

Method	Core idea
GD	full-dataset gradient — exact but slow
SGD	sample a batch — unbiased, noisy, cheap
Momentum	average gradients — smooth, accelerate
RMSProp	divide by RMS gradient — per-coord scale
Adam	momentum + adaptive scale + bias correction
LR schedule	big early, small late (warmup + decay)

Final mental model — one idea

Backprop answers: **what direction?**

Optimization answers:

how far, how smoothly, how adaptively?

gradient = direction · optimizer = the art of the step

Backprop gives the gradient; the optimizer turns it into learning.

Next: **regularization and generalization** — why the network that fits the training set also works on data it has never seen.